



**Universitat
de Lleida**



ESCOLA
POLITÀCNICA SUPERIOR
UNIVERSITAT DE LLEIDA

Homework Assignment: CBMC

Pràctica 2

6 de juny de 2026

May Castells Raga
Anna Marin Nuño

Sistemes de Verificació de Programari i Maquinari
Grau en Enginyeria Informàtica
Universitat de Lleida

Índex

1	Primer Problema: division.c	2
1.1	1.1 Anàlisi de l'Unwinding i Assumpcions	2
1.2	1.2 Verificació de l'Invariant i Localització del Bug	2
1.3	1.3 Cerca d'Errors de Desbordament (Overflow)	3
2	Segon Problema: simpleWhile.c	4
2.1	2.1 Determinació de l'Unwinding	4
2.2	2.2 Verificació de Requisits amb Assercions	5
2.3	2.2 Verificació d'accessos fora de rang (<i>pointer check</i>)	5
3	Tercer Problema: sort.c	6
3.1	3.1 Unwinding i Correcció de l'Assumpció	6
3.2	3.2 Anàlisi de temps de còmput	7
3.3	3.3 Introducció Controlada de Defectes	7

1 Primer Problema: division.c

1.1 Anàlisi de l'Unwinding i Assumpcions

El programa s'encarrega de calcular la divisió d'un dividend negatiu (D) entre un divisor positiu (d). Inicialment, les assumpcions del codi permetien que $d \geq 0$. Aquesta cota ens planteja un problema de verificació: si el divisor és zero, el residu no varia mai i el bucle esdevé infinit. Hem canviat l'assumpció a $d > 0$, una restricció que encaixa amb la definició de divisió per un nombre positiu.

Per triar un valor correcte d'unwinding (desenredament), hem de considerar el cas més desfavorable: $D = -600$ i $d = 1$. Aquí el bucle necessitarà 600 iteracions per arribar a un residu no negatiu.

És important entendre com funciona l'*unwinding assertion*: CBMC només considera que la verificació és exitosa si pot garantir que la condició de sortida del bucle es compleix (és a dir, la guard es nega) en algun moment dins del límit d'iteracions permès. Si la condició del bucle seguís sent certa després del desenredament màxim, l'assertió fallaria.

Si provem amb un valor de 600, veiem que l'eina ens avisa que el bucle encara podria requerir més passos:

```
# cbmc -no-standard-checks -unwind 600 -unwinding-assertions
division.c
```

Violated property:

```
file division.c function main line 10 thread 0
unwinding assertion loop 0: FAILURE
```

...

VERIFICATION FAILED

Amb el valor ajustat de 601, comprovem que mínim en una iteració (if del desenredament s'ha sortit del bucle:

```
# cbmc -no-standard-checks -unwind 601 -unwinding-assertions
division.c
```

VERIFICATION SUCCESSFUL

Per tant, el bucle realitza com a molt 600 iteracions, amb 601 la condició del últim desenredament serà falsa

1.2 Verificació de l'Invariant i Localització del Bug

Per assegurar la correctesa de l'algorisme, hem definit un invariant que s'ha de complir a cada inici de la iteració del bucle: $D = d \cdot q + r$. Fent servir l'opció `-trace`, podem observar la traça completa del contraexemple on es produeix la violació amb valors concrets:

```
# cbmc -no-standard-checks -trace -unwind 10 division_bug.c
```

** Results:

```
[main.assertion.1] line 9 Invariant: D == d*q + r: FAILURE
```

```

Trace for main.assertion.1:
State 11: D=-561, d=17, r=0, q=0
State 16: r=-561 (carreguem el dividend a r)
State 17: q=0
...
(Inici del bucle:  $r < 0$  es compleix)
(Condició  $r \leq -2*d$ :  $-561 \leq -34 \rightarrow$  CERT)
State 21:  $r=-527$  ( $r = r + 34$ )
State 23:  $q=-1$  (decrement del quocient:  $q--$ )

```

Violated property:

```

Invariant:  $D == d*q + r$ 
 $-561 == 17 * (-1) + (-527) \Rightarrow -561 == -544$  (FALS!)

```

En aquest contraexemple, el dividend és -561 i el divisor és 17 . L'invariant inicial es compleix ($D = -561, q = 0, r = -561$). En entrar al bucle, el programa detecta que pot aplicar el "salt doble" perquè el residu (-561) és menor o igual al doble del divisor negat (-34). Així doncs, suma 34 a r , obtenint -527 . Tanmateix, tal com mostra l'estat 23, el quocient només es decrementat en una unitat ($q = -1$). Al final de la iteració, l'invariant passa a ser $17 \times (-1) + (-527) = -544$, que difereix del dividend original -561 . Això ens confirma que per cada $2 \cdot d$ afegit a r , cal restar 2 a q .

Solució: Hem corregit aquest desequilibri fent que el decrement del quocient sigui consistent amb la quantitat sumada al residu. De manera que quan $r = r + (2*d)$, el quocient rebi $q = q - 2$:

```

1  if (r <= -(2*d)) {
2      r = r + (2*d);
3      q = q - 2;
4  } else {
5      ...

```

1.3 Cerca d'Errors de Desbordament (Overflow)

Un cop resolt el problema lògic, ens fixem si l'operació d'aritmètica pot desbordar el rang d'enters de 32 bits. Per fer-ho, primer demanem a CBMC que llisti les propietats de possibles desbordaments:

```
# cbmc -show-properties -signed-overflow-check division.c
```

Entre d'altres, trobem el risc de multiplicació a la línia on calculàvem $2 \cdot d$:

```

Property main.overflow.1:
arithmetic overflow on signed * in 2 * d
!overflow("?", signed int, 2, d)

```

Per comprovar aquest risc en el codi original, demanem a l'eina que analitzi exclusivament aquesta propietat:

```
# cbmc -signed-overflow-check -property main.overflow.1
division_original.c
```

**** Results:**

```
[main.overflow.1] line 11 arithmetic overflow on signed * in 2*d: FAILURE
VERIFICATION FAILED
```

Com s'observa, la propietat falla. Això es deu al fet que si d agafa valors molt elevats, el producte $2 \cdot d$ pot excedir la capacitat d'un enter de 32 bits, provocant un desbordament. Per tal d'evitar aquest risc mantenint el funcionament del programa, hem optat per substituir les operacions per sumes o comparacions equivalents que no generin desbordaments.

```
1 // Hem canviat r <= -2*d per r + d <= -d
2 // I el producte per dues sumes independents
3 if (r + d <= -d) {
4     r = r + d;
5     r = r + d;
6     q = q - 2;
7 } else {
8     r = r + d;
9     q = q - 1;
10 }
```

Amb aquestes modificacions, la verificació de desbordaments ens retorna (SUCCESSFUL) i el programa compleix les post-condicions finals de la divisió ($r \geq 0$ i $r < d$). Una altra possible solució seria usar l'algorisme de divisió clàssic, que es mostra a continuació:

```
1 int div(int D, int d) {
2     int r = D;
3     int q = 0;
4     while (r < 0) {
5         r = r + d;
6         q = q - 1;
7     }
8     return q;
9 }
```

2 Segon Problema: simpleWhile.c

L'objectiu en aquest segon exercici és comprovar la correctesa d'un programa que vol buscar el nombre parell màxim dins un *array* generat aleatòriament. La mida o llargària fixada pels enters d'anàlisi és constant de `#define N 10`.

2.1 2.1 Determinació de l'Unwinding

Atès que estem treballant amb bucles `for` amb un límit fix definit per $N = 10$, el nombre d'iteracions necessari és senzill de calcular.

Si provem amb un desenredament de $K = 10$, l'eina ens indica que encara hi ha possibles iteracions pendents:

```
# cbmc -no-standard-checks -unwind 10 -unwinding-assertions
simpleWhile.c
```

```
[main.unwind.0] line 10 unwinding assertion loop 0: FAILURE
VERIFICATION FAILED
```

Per tant, utilitzem 11 per a veure si amb 11 se surt del bucle:

```
# cbmc -no-standard-checks -unwind 11 -unwinding-assertions
simpleWhile.c
```

VERIFICATION SUCCESSFUL

Per tant, el bucle realitza com a molt 10 iteracions i seria suficient.

2.2 Verificació de Requisits amb Assercions

Un cop finalitzada l'execució del bucle, hem afegit les següents assercions per validar que el programa compleix els requisits esperats:

- (a) Si hi ha components parells, llavors *maxeven* conté el més gran: Això es comprova verificant que *maxeven* existia a la llista, és efectivament un nombre parell i, per a cada component *i* parell dins de l'array, aquest és menor o igual a *maxeven*.
- (b) Quan *maxeven* és exactament 1, tots els enters a l'array són senars: Iterarem comprovant que residu en div per 2 és $\neq 0$.

El codi integrat dins de 'simpleWhile.c' queda representat com:

```
1  if (maxeven != 1) {
2      for (i = 0; i < N; i++) {
3          if (numbers[i] % 2 == 0) {
4              __CPROVER_assert(maxeven >= numbers[i],
5                  "maxeven és >= que qualsevol parell");
6          }
7      }
8      int found = 0;
9      for (i = 0; i < N; i++) { if (numbers[i] == maxeven) found =
10         1; }
11     __CPROVER_assert(found == 1, "maxeven existeix a l'array");
12     __CPROVER_assert(maxeven % 2 == 0, "maxeven és parell");
13 }
14 else {
15     // No hi ha nombres parells: tots són senars
16     for (i = 0; i < N; i++) {
17         __CPROVER_assert(numbers[i] % 2 != 0, "nombres senars");
18     }
19 }
```

2.3 2.2 Verificació d'accessos fora de rang (*pointer check*)

Cal verificar que no hi hagi errors d'accés fora de rang (*out of bounds*) en les línies que operen amb l'array *numbers*. Per dur a terme aquesta comprovació, fem servir l'opció *-pointer-check* de CBMC.

```
# cbmc -no-standard-checks -unwind 11
-unwinding-assertions -pointer-check simpleWhile.c
```

```

** Results:
...
VERIFICATION SUCCESSFUL

```

Amb aquest resultat (SUCCESSFUL), CBMC ens dona la certesa que totes les operacions d'accés al vector `numbers[i]` es troben dins dels límits del buffer $[0, N - 1]$ i no es produeix cap violació de segment ni accés a memòria no autoritzada.

3 Tercer Problema: sort.c

En l'últim exercici implementem l'algorisme d'ordenació de la *Bombolla* (*Bubble Sort*) per ordenar una cadena d'enters de petit a gran. Aquest algorisme funciona comparant repetidament parelles d'elements adjacents i intercanviant-los si estan en l'ordre incorrecte. En cada passada completa per l'array, l'element més gran que encara no està al seu lloc es trasllada cap a la seva posició definitiva al final de l'estructura.

3.1 3.1 Unwinding i Correcció de l'Assumpció

Per a determinar l'unwinding necessari, hem de fixar-nos en tots els bucles del programa. Tot i que els bucles de l'algorisme *sort* iteren $N - 1$ vegades, el bucle d'inicialització a la funció `checkSort` itera exactament N vegades. Per tant, per satisfer l'*unwinding assertion* global, necessitem un valor de $K = N + 1$.

Per exemple, si usem $N = 8$, provar amb $K = 8$ ens retornarà un error de verificació perquè el bucle d'inicialització encara no hauria confirmat la seva sortida:

```
# cbmc -no-standard-checks -unwind 8 -unwinding-assertions sort.c
```

```

[checkSort.unwind.0] line 33 unwinding assertion loop 0: FAILURE
VERIFICATION FAILED

```

(Nota: la línia 33 correspon al bucle `for (i = 0; i < N; ++i)` d'inicialització).

En canvi, amb $K = 9$, la verificació es completa sense problemes:

```
# cbmc -no-standard-checks -unwind 9 -unwinding-assertions sort.c
```

```
VERIFICATION SUCCESSFUL
```

Cal esmentar que el codi original incloïa una assumpció incorrecta: `array[i] <= -20`. Atès que això entrava en conflicte amb els rangs previstos (deixant l'espai de cerca gairebé buit), ho hem esmenat per `array[i] >= -20`, assegurant un interval de treball vàlid entre -20 i 20.

Per tant, cadascun dels bucles `for` realitza com a molt 8 iteracions $K=8$ i tenim aquesta certesa gràcies a que la segona comprovació confirma que no hi havia cap execució de 9 iteracions.

3.2 Anàlisi de temps de còmput

Amb CBMC es crida especificant $\text{unwind} = N + 1$:

```
cbmc -no-standard-checks -unwind <N+1>
-unwinding-assertions sort.c
```

Presentem en la següent taula els temps executats amb la versió 6.9.0 de CBMC

N	4	6	8	10	12
time	0.100s	0.146	0.282	0.694	1.933

Taula 1: Temps real en verificació de CBMC de la funció sort per la variable array size (N).

L'increment del temps és realment exponencial?

D'acord amb les dades recollides a la taula, s'aprecia com el temps augmenta de manera molt més ràpida que la mida de l'array. Per exemple, passar de $N = 10$ a $N = 12$ (un increment molt petit d'elements) gairebé triplica el temps de computació. Aquesta tendència és característica del *bounded model checking*: en incrementar el nombre d'elements i d'iteracions, el programa desenredat resultant creix en complexitat i el nombre de variables i clàusules que el solucionador SAT ha de gestionar es dispara. Aquest fenomen, conegut com l'explosió de l'espai d'estats, fa que la verificació completa de llistes grans sigui ràpidament intractable pel que fa a recursos de temps i memòria. El creixement no és exponencial però s'hi acostava.

3.3 Introducció Controlada de Defectes

Per provar a trobar errors, hem introduït un error simple en la implementació de l'ordenació: fer que un dels bucles acabi abans d'hora ($i < \text{size} - 2$). Això provoca que l'algorisme no completi l'última passada necessària per garantir l'ordre total.

Per il·lustrar l'error, hem utilitzat un cas amb $N = 4$ i l'entrada desordenada $\{1, 1, 2, 0\}$. A continuació presentem l'evolució dels estats segons CBMC:

```
State 30: array={ 1, 1, 2, 0 } (Càrrega de l'entrada inicial)
...
(Passada i=0, j=2): es compara array[2]=2 amb array[3]=0.
Com que 2 > 0, s'executa el SWAP:
State 60: array[2]=0
State 61: array[3]=2 (L'array ara és { 1, 1, 0, 2 })
...
(Passada i=1, j=1): es compara array[1]=1 amb array[2]=0.
Com que 1 > 0, s'executa el SWAP:
State 84: array[1]=0
State 85: array[2]=1 (L'array ara és { 1, 0, 1, 2 })
...
(Final del bucle extern: i=2. Condició i < size-2: 2 < 2 -> FALS)
El bucle s'atura precipitadament.
```

Violated property:

```
Array is not sorted (array[0] <= array[1] -> 1 <= 0) : FAILURE
```


contraexemple: L'algorisme de la bombolla necessita $N - 1$ passades per garantir l'ordre en el pitjor dels casos. Amb $N = 4$, caldrien 3 passades (per $i = 0, 1, 2$). Començant amb $\{1, 1, 2, 0\}$, després de la primera passada ($i = 0$) el 2 arriba al final. En la segona ($i = 1$), el 0 avança una posició cap a l'esquerra. En aquest punt l'array és $\{1, 0, 1, 2\}$. El bug que hem injectat ($i < \text{size} - 2$) fa que el bucle ignori la darrera passada necessària. Així, quan CBMC verifica la propietat d'ordenació, troba immediatament que el primer element (1) és més gran que el segon (0). Aquest contraexemple prova que l'algorisme és incorrecte.